kubectl get pods --all-namespaces (to check runnint pods)

Here's an example .yaml file that shows the required fields and object spec for a Kubernetes Deployment:

application/deployment.yaml

```
apiVersion: apps/v1 # for versions before 1.9.0 use apps/v1beta2
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2 # tells deployment to run 2 pods matching the template
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

kubectl create service nodeport nginx --tcp=80:80

servicefile.yml

```
apiVersion: v1

kind: Service

metadata:

  name: my-service

spec:

  selector:
```

```
      app: nginx
  type: NodePort
  ports:
   - protocol: TCP
     nodePort: 31000
     port: 80
     targetPort: 80
```

https://kubernetes.io/docs/tasks/configure-pod-container/

kubectl create service nodeport nginx --tcp=80:80

Replicationsets:

Kubectl create -f replicaset-definition.yml

kubectl get replicaset

kubectl delete replicaset myapp-replicaset

kubectl replace -f replicaset-definition.yml

kubectl scale --replicas=6 -f replicaset-definition.yml

Deployment:

Rollout - new deployment with revision1, if update the image then its Revision2

kubectl rollout status deployment/myapp-deployment

DS -

Recreate - Destroy all and create new instances (Application will go down)

Rolling Update - one after one

kubectl apply -f deployment-definition.yml

Rollback:

kubectl rollout undo deployment/myapp-deployment

https://kubernetes.io/docs/tasks/configure-pod-container/assign-pods-nodes/ **(to specify nodes)**

kubectl run nginx --image=nginx --port=80 deployment "nginx" created

sudo swapoff -a

sudo sed -i '/ swap / s/^/#/' /etc/fstab

Reboot a machine after that.

kubeadm reset

kubeadm init --ignore-preflight-errors all